Mobile robot localization in indoor environments: Fusion of odometric data and BLE trilateration via Kalman filter

DEROCHE Florent, DEFFAYET Jolan, GONCE Loïs, YANG Kaiyue

April 10, 2026

Abstract

The localization of mobile robots in indoor environments is a major challenge for Industry 4.0, given that satellite signals (GNSS) are ineffective there. This article presents a low-cost and easily deployable solution to address this industrial problem. This system relies on the trilateration of Bluetooth Low Energy (BLE) signals based on RSSI, emitted by three ESP-32s acting as anchors, coupled via a real-time Kalman filter with odometric data from the mobile robot's wheels. This Kalman filter is implemented under the ROS2 framework, and communication between the TurtleBot's ESP-32 and the computer receiving the data is handled by an MQTT server. Our experiments show that this filtering successfully corrects wheel drift while smoothing out positioning errors related to BLE. The system achieves an accuracy of approximately 4 meters. To guarantee accessibility for everyone, all the code and procedures are freely available on the project's GitHub page.

1 Introduction

The precise localization of mobile robots in indoor environments constitutes a fundamental challenge in robotics. Unlike outdoor environments, where global navigation satellite systems (GNSS) offer centimetric precision, confined spaces render these technologies inoperative due to satellite signal attenuation. This limitation requires the use of alternative approaches. Among the available technologies, Bluetooth Low Energy (BLE) is of particular interest for indoor localization. Its low energy consumption, simplified deployment, and widespread presence in embedded devices make it an ideal candidate for low-cost localization systems. However, Received Signal Strength Indicator (RSSI) measurement suffers from significant uncertainty due to electromagnetic reflections and interferences in indoor environments, resulting in a noisy and unstable estimation.

To correct these limitations, the fusion of complementary information sources is a robust approach. Wheel encoder odometry, natively available on most mobile robots, offers precisely the necessary complementarity: highly accurate locally and at high frequencies, it nevertheless suffers from a cumulative drift that makes it inaccurate in the long term. Fusing these two modalities within a Kalman filter then allows exploiting their respective strengths. Indeed, the absolute BLE correction compensates for the odometric drift, while the odometric prediction smooths out the discontinuities of the BLE measurement. This article presents the implementation and evaluation of such a system on a TurtleBot Kobuki robot, relying on an ESP32-based hardware infrastructure and a ROS2 software architecture, with all source code made freely available on the project's GitHub repository.

The remainder of the article is organized as follows: Section 2 details the hardware and software architecture as well as the methodological choices; Section 3 describes the experimental protocol; Section 4 presents and discusses the results; Section 5 concludes this experiment.

2 System Architecture and Methodology

2.1 Technological choices

The chosen architecture relies on a selection of standard technologies in the field of embedded robotics and IoT, favoring availability, low cost, and native integration.

- Bluetooth Low Energy (BLE): A low-energy wireless communication protocol, used as the primary measurement tool. The BLE beacons periodically broadcast a signal whose received power is exploited to estimate the distance between the robot and each anchor.
- ESP32: An Espressif microcontroller natively integrating BLE and Wi-Fi on the same chip. It is used both for the fixed anchors that broadcast a configurable-power BLE signal and for the module attached to the robot that scans the signals from the anchors. It applies an EMA filter on the received signal strengths (RSSI) and transmits the measurements to the MQTT broker via Wi-Fi. Its low cost makes it the main component of the hardware infrastructure.
- MQTT Mosquitto: A lightweight messaging protocol based on TCP/IP, designed for unreliable networks or networks with limited bandwidth resources. It is primarily used for machine-to-machine (M2M) communications or Internet of Things (IoT) connections. The robot publishes the filtered RSSIs of the three anchors as a JSON message. The Mosquitto broker, deployed on a machine connected to the IMT Nord Europe IoT network, relays these messages to the ROS2 trilateration node.
- ROS2: An open-source robotics framework constituting the high-level processing layer. Its modular architecture of communicating nodes via topics, its TF2 coordinate frame transformation system, and its RViz2 visualization tool facilitate the integration of different data sources. Two Python nodes were developed: `trilateration_node` for the BLE position calculation, and `fusion_node` for the Kalman fusion with the Kobuki robot's odometry.
- Kalman Filter: A recursive estimation algorithm combining prediction and dynamic correction. It is used to fuse the two localization modalities operating at different frequencies: the odometric prediction ensures the continuity of the estimation, while the BLE correction compensates for the cumulative drift of the odometry. The filter is parameterized by a process noise matrix $\mathbf{Q}$ reflecting the confidence in the odometry, and a measurement noise matrix $\mathbf{R}$ reflecting the uncertainty of the BLE estimations.

2.2 Hardware and software architecture

The architecture of our localization system mainly consists of a hardware component and its embedded firmware, as well as a software component using our ROS python scripts and an MQTT server running on a host PC. Regarding the hardware, we have 4 ESP32s with integrated antennas, 3 portable batteries, a Kobuki robot carrying a Raspberry Pi and various sensors (Odometry, Infrared Camera, etc.), and finally a host PC running the MQTT Broker on Ubuntu with ROS2 installed.

- Initially, 3 ESP32s are dedicated to operating as static "Anchors" powered by portable batteries, transmitting BLE signals at well-known and measured locations in the environment where the robot operates.
- Secondly, another ESP32 is installed and fixed on the hood of the Kobuki robot, intended to operate as a measuring "station" for the signal power emitted by each Bluetooth Anchor, and to communicate the results to the MQTT server via Wi-Fi.
- Finally, the RSSI measurement results are received by the MQTT server and are then transmitted to our ROS2 Python packages, which perform the trilateration calculations, data fusion, as well as the collection of odometric data from the Kobuki's wheels.

At the embedded software level, the C/C++ firmwares of the "Anchor" and "Robot" ESP32s were programmed and flashed onto these microcontrollers using the Arduino development environment provided by the "PlatformIO" tool and the VS Code IDE. The main libraries used are the standard BLE library, as well as PubSubClient and ArduinoJson for sending JSON messages to the MQTT server. Therefore, to bridge the ESPs, the Kobuki robot, and our Ubuntu PC for the ROS scripts, we used the Mosquitto broker. An important point is that all devices must be connected to the same

Wi-Fi network with the correct IP address configured so that communication occurs properly.
At the ROS package level, we have a launch file to start our 3 main nodes: "ble localization", "fusion node", and "ble origin broadcaster".

- "ble localization" retrieves the RSSI data via Json messages from the MQTT broker and performs the bluetooth trilateration by solving the position equation using the least squares method:
$$d_i = \sqrt{(x - x_i)^2 + (y - y_i)^2 + (z - z_i)^2} \tag{1}$$
where x, y are the coordinates to calculate for our robot, z is the known height of the robot's ESP32, and x_i, y_i, and z_i are the coordinates of our anchors.
- "fusion node" retrieves this initial position estimation solely via bluetooth, and fuses this data with the odometry from the Kobuki robot's wheels using a 2D Kalman filter to smooth our results.
- "ble origin broadcaster" simply publishes a static frame in our map, at the coordinates of our real reference point (Origin), at the robot's starting location.

2.3 BLE signal modeling

The Log-Distance Path Loss Model, which establishes a relationship between the received BLE signal power and the distance between the transmitter and receiver, is used to estimate the distance separating each anchor from the robot.

$$RSSI(d) = RSSI(d_0) - 10n \log_{10}\left(\frac{d}{d_0}\right) + X_\sigma \tag{2}$$

Where:

$RSSI(d)$ Received signal power at distance d;

$RSSI(d_0)$ Reference power measured at a known distance d_0;

n Path loss exponent;

X_σ Noise.

2.4 Data Fusion via Kalman Filter

To reliably and continuously estimate the position of the Kobuki robot, we implemented a data fusion architecture based on a 2-dimensional Kalman filter. The goal is to smooth the bluetooth position obtained every 500 ms with the wheel odometry, which is retrieved at a very high frequency with good precision. The robot's odometry forms the basis of our prediction step: it is effective for smooth short-term movements but suffers from cumulative drift over time.
Conversely, our Bluetooth trilateration system acts as a correction step by providing absolute coordinates in the "ble origin" room frame. However, since the radio signal is inherently unstable, these raw measurements exhibit significant noise.

- **Innovation (Measurement Error):**
$$Y_k = Z_k - \hat{X}_k^-$$
- **Innovation Covariance:**
$$S_k = P_k^- + R$$
- **Kalman Gain:**
$$K_k = P_k^- S_k^{-1}$$
- **State Update (Fused Position):**
$$\hat{X}_k = \hat{X}_k^- + K_k Y_k$$
- **Covariance Matrix Update:**
$$P_k = (I - K_k) P_k^-$$

The major challenge lies in calibrating the filter with coherent values for the covariance matrices Q and R, to avoid large jumps in real-time position estimation. (Here Q is minimal, and R is very high)

2.5 Data Retrieval and Analysis

To evaluate the performance of the methods used, they must be compared synchronously with the exact positions generated by the SLAM. To do this, an acquisition script and an analysis script were implemented.
Data recording is handled by a dedicated ROS2 node developed in Python. This node subscribes to both the raw positioning (BLE trilateration) and filtered positioning (Kalman filter) topics. Simultaneously, the robot's exact position is extracted from the ROS2 transformation tree. Then, the script projects the positions into a common frame. Finally, the positions are exported to a CSV file.
A second script analyzes the extracted data and calculates the Mean Absolute Error (MAE), the Root Mean Square Error (RMSE), calculated with the formula below, as well as the average *jitter*, which quantifies the shaking of the estimated position from one moment to the next.

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2} \tag{3}$$

3 Experimental Protocol

3.1 Description of the experimental environment

To evaluate the performance of the localization system, an experiment was conducted in an indoor environment. The test area is a room with an area of 50m² (10m by 5m). This environment includes various obstacles such as floor obstacles hindering the robot's mobility and concrete walls preventing the reception of GNSS signals.

3.2 Reference localization method

To evaluate the accuracy and reliability of the localization system, the obtained results must be compared against a reliable estimated reference position. To obtain these reliable positions, a system based on LiDAR technology coupled with a Simultaneous Localization And Mapping (SLAM) approach is used. The LiDAR equipped on the robot directly generates highly accurate geometric data. During movement, the sensor scans obstacles and the SLAM algorithm builds a map of the environment in real-time while calculating the robot's exact position within this map. We thus obtain a reference positioning with centimetric precision.

3.3 Methodology and execution of the experiment

This section details the implementation of the localization system. To ensure the reproducibility of the results, an experimental protocol was defined. This revolves around preparing the experimental area followed by a data acquisition phase during which the robot moves.

For this, here is the list of required equipment:

- 4 ESP32 DevKit C-V4
- 3 power banks
- 4 USB-C to micro USB cables
- 1 Kobuki Turtlebot equipped with LiDAR technology
- 1 computer
- 1 tape measure
- 1 controller

Here are the steps to follow:

1. initialize the MQTT server
2. flash 3 ESP32s that will act as anchors with the appropriate code

3. flash the ESP32 that will go on the robot
4. connect the 3 ESP32s serving as anchors to their power banks
5. define an origin point in the room to serve as a starting point for the robot
6. define an origin orientation for the robot
7. place the anchors in the environment
8. record the coordinates of the anchors
9. connect an ESP32 to the robot
10. place the robot at the origin point in the orientation defined previously
11. turn on the robot and the anchors
12. measure the RSSI of the anchors at 1m and input the obtained value into the code
13. launch the SLAM algorithm
14. launch the ROS2 nodes for position estimation (BLE alone and Kalman filter) and data retrieval
15. move the robot to gather relevant data

4 Results and Discussion

4.1 Analysis of the results

The following results are obtained:

Estimation Method	**MAE (m)**	**RMSE (m)**	**Average Jitter (m)**
Raw BLE Trilateration	3.929	4.064	0.235
Kalman Filter	3.785	3.921	0.044

Table 1: Comparison of positioning errors relative to the exact position (SLAM).

We see that the Kalman filter provides a more precise estimation; a 3.6% improvement is obtained with the Kalman filter on the MAE and a 3.5% improvement on the RMSE. These improvements are minimal, but the addition of odometry provides better spatial stability, as can be seen with the average jitters. Indeed, we obtain an 81.3% improvement in the average jitter.

4.2 Limitations and areas for improvement of the system

Although the Kalman filter improves the result, an estimation error of 4 meters remains high, and this is inherent to the technology used. RSSI is very sensitive to reflections off walls and obstacles, making the estimation highly unstable even when the robot is stationary.

As an area for improvement, adding more anchors would improve accuracy since three anchors are the minimum required for trilateration, but in case one of these anchors is obstructed, the geometric calculations will be distorted. Using another localization method such as "fingerprinting" with a neural network could also be a viable improvement track. It would be possible to measure Bluetooth at numerous points throughout the room, along with the exact positions via SLAM, and then create datasets to train the neural network, which would thus find position estimations by "learning" the reflection of the Bluetooth signal.

5 Conclusion

This work presented a complete indoor localization system without GNSS infrastructure, combining BLE trilateration and wheel encoder odometry within a 2D Kalman filter, deployed on a Kobuki robot via the ROS2 framework. The proposed architecture, based on low-cost components (ESP32, BLE, MQTT), is functional and demonstrates the technical feasibility of the approach in a real environment.

The experimental results accurately quantify the system's contributions and limitations. BLE trilateration alone achieves a MAE of 3.929 m, an RMSE of 4.064 m, and an average jitter of 0.235 m. The addition of the Kalman filter reduces these values to 3.785 m, 3.921 m, and 0.044 m, respectively. The 81.3% reduction in jitter constitutes the most significant result of this study. It demonstrates that odometric fusion brings spatial stability: the discontinuities and instabilities of the raw BLE trilateration are almost entirely absorbed by the odometric prediction, producing a more stable and regular estimated trajectory.

Finally, this work confirms that the fusion of BLE trilateration and odometry via a Kalman filter constitutes an accessible and deployable low-cost approach for the indoor localization of mobile robots. While the absolute precision obtained remains insufficient for autonomous navigation, the results validate the architectural principle and identify levers for improvement: adding anchors, calibrating the propagation model, and utilizing neural network learning methods. These directions constitute the future perspectives of this work.

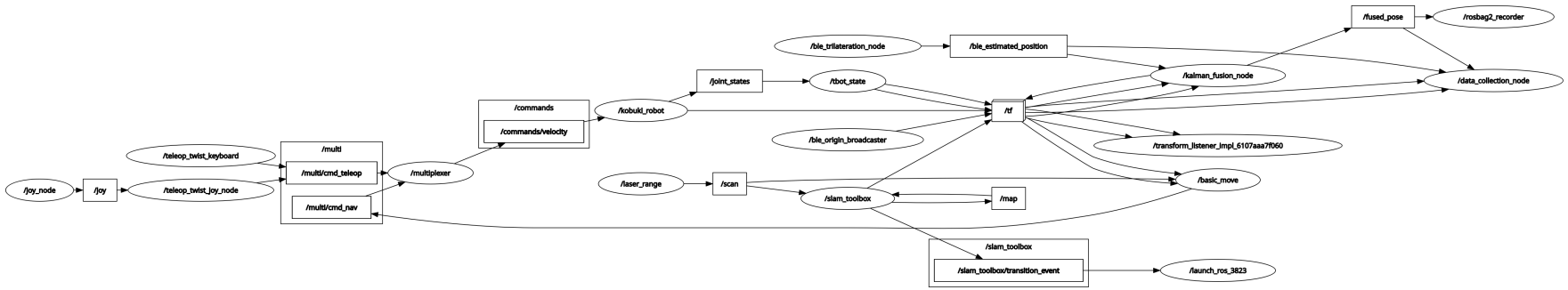

Figure 1: ROS communication graph (rqt_graph)